

Data Pipelines Platform for Analytics and AI: Proof of concept (POC) - Bring your own use case

Data Quality and Decision Urgency of the use case drive pipeline architectures for high-quality and timely decision making

Pradeep Kalluri and Raghu Shankar
July 2026

Summary

Data is the foundation for many of decisions made collaboratively by humans and AI. Data quality and decision urgency are becoming more critical as more decisions are transitioned to AI. This paper and associated proof of concept (POC) platform covers three independent data pipeline architectures to deliver high quality data at required levels of urgency for enterprises tackling data pipelines for human and AI collaborative analytics, decisions and actions. This POC is built on proven open-source and cloud-native infrastructures including Kubernetes, Docker, Cilium, PostgreSQL, ClickHouse, Apache Iceberg, MinIO object store S3, Apache Airflow, data build tool (dbt), natural language queries via MCP, Spark, Debezium, Apache Kafka, Strimzi, KRaft, Prometheus, and Grafana.

We've deployed a functional platform POC on cloud-native with observability for sandbox testing of use cases. Refer to github site: <https://github.com/kalluripradeep/modern-etl-stack>. We **invite** interested parties to try it in a sandbox, contribute, or provide feedback for improvements.

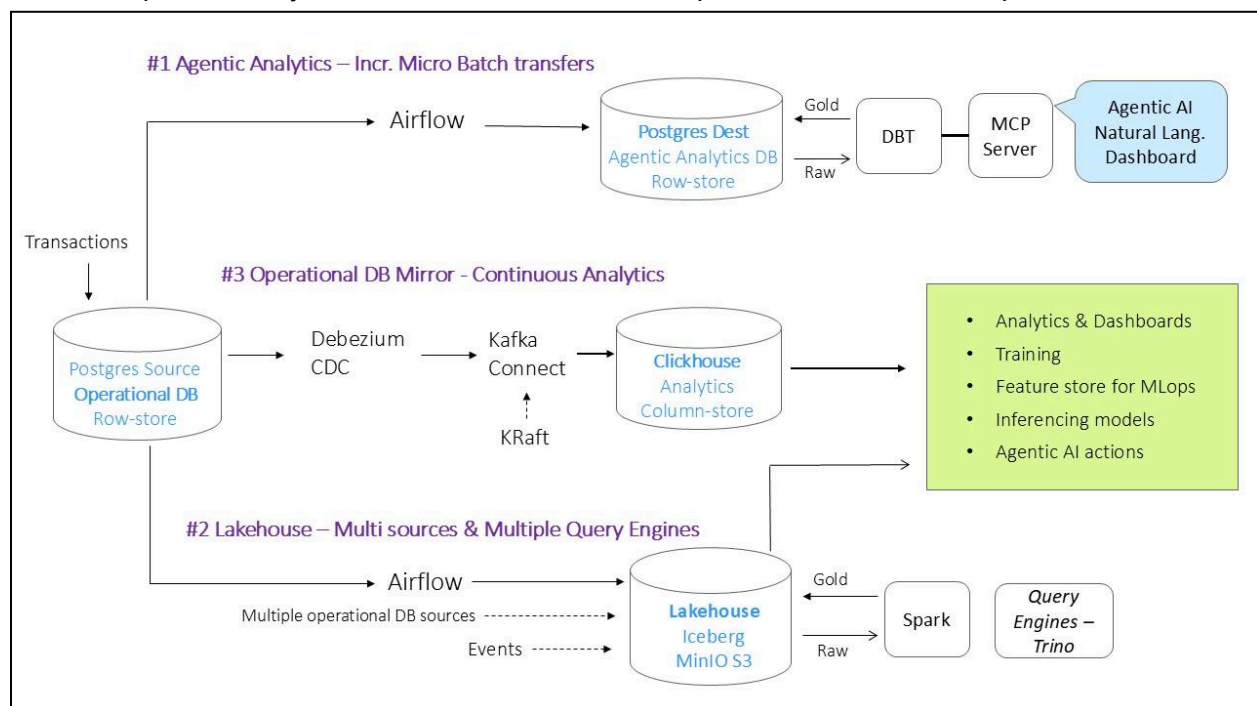


Table of Contents

Summary	1
1. Introduction	3
2. Data Pipelines	4
2.1 Data Pipe #1: Agentic AI Analytics - Incremental micro-batch data transfers	4
2.2 Data Pipe #2: Lakehouse - Multiple sources and Multiple Query Engines	4
2.3 Data Pipe #3: Streaming CDC Mirror - Continuous Analysis	5
3. Natural Language Analysis	5
3.1 Zero-Hallucination Architecture:	5
3.2 Autonomous Validation:	5
3.3 Human-in-the-Loop Self-Healing:	5
4. Cloud Native deployment with persistent storage	5
5. Infrastructure for Data Pipe #1 Agentic Analytics - Incremental micro-batch data transfers	6
5.1 Postgres operations and agentic analytics databases	6
5.2 Apache Airflow	7
5.3 dbt - Data Build Tool	9
5.4 Natural Language Queries	9
6. Infrastructure for Data Pipe #2 Lakehouse - Multiple Sources and multiple Query Engines	10
6.1 Apache Iceberg and MinIO	10
6.2 Apache Spark 3.5	11
7. Infrastructure for Data Pipe #3: Operational CDC Mirror - Continuous Analysis	12
7.1 Kafka Confluent	12
7.2 Kafka Connect and Debezium	13
7.3 Strimzi	13
7.4 KRaft	13
7.5 ClickHouse	13
8. Observability for pipelines	14
8.1 Node Exporter	14
8.2 Cilium and Hubble	14
8.3 Prometheus	14
8.4 Grafana	15
9. Security	15
10. Test case and Use case	15
11. Summary and Next Steps	16
Authors	17
References & Related Readings	18

1. Introduction

Data is the foundation for analytics, AI, insights, decisions, and actions. Until recently humans were responsible for most of the **decisions** and consequent **actions** based on analytics. Lately with the advent of AI, the balance is shifting more towards AI to process data, uncover insights, generate decision choices with impacts, pros and cons, and in few instances act (or ignore) the decisions. Data is also used for training AI models, inferencing, generative AI, and agentic AI.

AI systems can process data, create decision choices, and act on it (if authorized) far faster than humans. Data quality and decision urgency become even more imperative for time-sensitive analysis, high quality decisions, and actions executed collaboratively by humans and AI. Many other factors of governance, lineage, workflows, and trust are critical but out of scope in this paper.

This paper covers three architecture patterns of high relevance to enterprises tackling data pipelines. First data can be categorized by two key attributes - **quality** and **urgency** of decisions of the data. Second the organization's process and use cases addressed provides a choice of analytics database - either Postgres or lakehouse.

Three cloud-based data pipelines are described around these requirements. We've built a proof of concept (POC) as a platform to demonstrate the architecture patterns. Its deployment, basic functional testing, and observability is complete on a local Kubernetes cluster. We've picked open-source for cost and avoiding vendor lock-ins. However this can also be deployed in Kubernetes public or private clouds after configuring the deployment scripts. Refer to github site: <https://github.com/kalluripradeep/modern-etl-stack>. This POC is a follow-up to the paper by Pradeep Kalluri titled "From Raw to Refined: Data Pipeline Architecture at Scale," [Ref 1, [Link](#)].

We invite interested parties, especially **data engineers** and **data platform engineers**, to try it in a sandbox, contribute via push requests (PR), or provide feedback via github **Issues** for improvements. We'll discuss the infrastructure elements deployed for this proof of concept and reasons for their inclusion. A single pipelines.yml manifest drives the ingestion Directed Acyclic Graphs (DAG), Debezium connector, CDC topics, and ClickHouse schema, with Continuous Integration (CI) blocking any drift. **Integrate domain-specific data tables** for testing your use case.

One test case has been created for basic functional testing. This test generates data traffic that exercises all three pipelines. In future phases, more extensive test cases will be created. Second sample use cases in banking, finance, health, life sciences, manufacturing, and logistics will be tested.

2. Data Pipelines

Pipelines use proven modern infrastructures (refer to figure): Airflow for complex orchestration, Spark for large datasets processing, DBT for transformations, Kafka for messaging, and MCP server for natural language queries. Business Intelligence (BI) tools for humans and agentic AI tools can be hooked up but will be covered in future work.

One operational source Postgres database feeds three **independent**, parallel pipelines: (1) Micro-batch for agentic AI, (2) Iceberg lakehouse, and (3) Streaming CDC mirror [Ref 2].

2.1 Data Pipe #1: Agentic AI Analytics - Incremental micro-batch data transfers

Ideal for human-centric natural language queries for decision making and relative operational simplicity. The objective is to off-load data periodically from operational databases to an analytics database, thereby maintaining the performance SLAs of the operational database. A natural language query engine allows business analysts and domain experts to immediately query data without depending on data analysts to write structured query language speeding up decisions by experts.

This data transfer is managed by Airflow. Snapshots are extracted periodically from destination Postgres and DBT transforms into an analytics schema (Gold layer). This provides high data quality and near-fresh data for agentic AI including natural language queries. It also applies when processing large historical volumes but data transfer resources are minimal. Advantages are simpler architecture for deployment and debugging and lower infrastructure resource consumption.

2.2 Data Pipe #2: Lakehouse - Multiple sources and Multiple Query Engines

Apache Iceberg is used as a lakehouse for ingesting data from operational Postgres. This pipeline is ideal when ingesting data from multiple operational databases, via streaming or batch, as well as data from asynchronous events that affect decisions. Second, multiple groups need to query the lakehouse for analytics via diverse query engines. A lakehouse maintains data integrity and hence the quality of data. Supporting multiple input sources simultaneously while making data available for multiple agents addresses decision urgency without blocking any specific entity.

This pipe transfers data in chunked micro-batches from operational Postgres to Apache Iceberg lakehouse. This transfer is managed by Airflow and chunking saves memory. Spark processes data in Apache Iceberg tables (Silver layer) in MinIO. This pipe is designed to handle billions of rows that would be too expensive to store and process for analytics in an operational database. Apache Iceberg This pipe offers a balance of decision urgency while maintaining high data access by multiple entities.

2.3 Data Pipe #3: Streaming CDC Mirror - Continuous Analysis

Captures real-time data changes using change data capture (CDC) with Debezium and Kafka. This provides a continuous mirror of the source data in Clickhouse store for live dashboards and operational monitoring while reducing performance impact on the operational database. Moreover near-real time data feeds to AI inference models allows for high urgency decisions and actions by humans and AI agents.

3. Natural Language Analysis

The Postgres destination database in data pipeline #1 features state-of-the-art Model Context Protocol (MCP) integration, allowing LLM coding agents (like Claude Desktop or Cursor) to act natively as data engineers [Ref 4]. Through the custom dbt-mcp server located in this repository, the AI Agent interacts directly with data from the production environment via replicated mirror data [Ref 3 and 4].

3.1 Zero-Hallucination Architecture:

The Agent explicitly reads and writes real SQL code and schema metadata directly from the Destination Data lakehouse. It never has to "guess" or "hallucinate" table structures because it has live, native database access.

3.2 Autonomous Validation:

It can natively trigger a dbt test against the live PostgreSQL database to instantly verify its own code changes.

3.3 Human-in-the-Loop Self-Healing:

By hooking the MCP directly into the lakehouse, the agent can analyze live pipeline failures and explicitly write and test its own SQL patches. However, it strictly requires human approval before any code is committed or applied, guaranteeing complete control and security without requiring human copy-pasting.

4. Cloud Native deployment with persistent storage

Cloud native technologies enable building and running scalable applications in private and hybrid clouds. Containers, service meshes, microservices, and declarative resources exemplify this approach. These enable loosely coupled systems that are resilient, manageable, and observable (Refer <https://www.cncf.io/>).

Many applications and databases are cloud-native which simplifies deployment and monitoring in Kubernetes environments. Importantly many have built Kubernetes **operators** that make deploying and managing their resources relatively simpler in Kubernetes environments. These resources can be configured, deployed, managed, connected, and scaled like any native

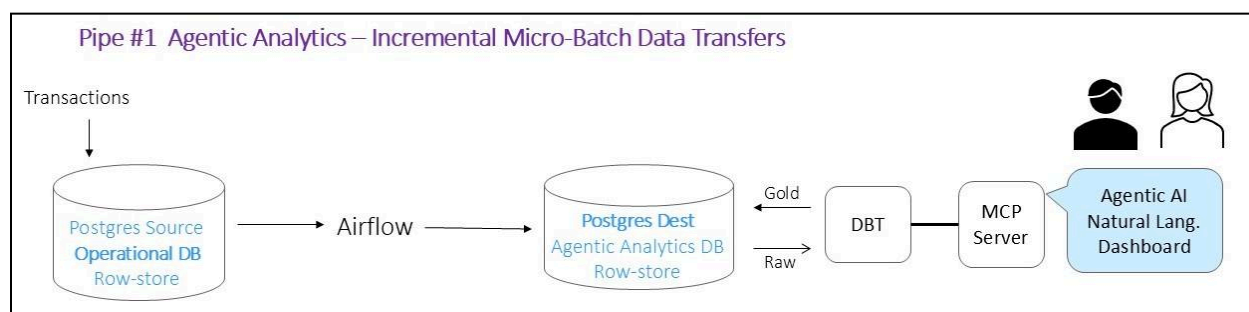
Kubernetes resource. There is **disaggregation** of compute and storage where each can be scaled as needed.

Kubernetes also helps with lifecycle management of storage resources required for many infrastructure elements of data pipelines. Storage classes are created with Rancher that auto provisions persistent capacity on local disk storage. Kubernetes handles the requests from pods to dynamically create their storage claims.

This proof of concept has been tested on Kubernetes cluster consisting of two bare metal servers each with 40 cores, 64GB memory, local disk storage, Ubuntu, and Kubernetes control plane and worker nodes.

[Cilium](#) and [Hubble](#) have been installed in the test cluster. Cilium is an open source, cloud native eBPF-based solution for providing security and observability of network connectivity for workloads. Hubble, enabled via Cilium, collects network flows and bare metal metrics and forwards them to Prometheus. Observability can be set up via Kubernetes node-ports accessed via web browser. Grafana standard dashboards for Kubernetes cluster monitoring can be imported from [Kubernetes Cluster Monitoring via Prometheus - ID 315](#). This shows CPU, memory, and IO, along with container and pod metrics.

5. Infrastructure for Data Pipe #1 Agentic Analytics - Incremental micro-batch data transfers



This pipe utilizes Postgres as a destination database. Data is transferred via micro-batches typically 30-min intervals. This opens up urgent analysis on high quality data directly to business analysts and business group experts. It offers relative operational simplicity and lower capex by leveraging Postgres skill sets in the organization. dbt is used to improve data quality. MCP server is connected to dbt for agentic AI analytics [Ref 4].

5.1 Postgres operations and agentic analytics databases

Postgres has established itself as one of the top 5 databases and has been widely deployed for decades across the industry. Importantly Postgres is a thriving ecosystem that supports extensions allowing addition of capabilities independent of the core platform - with no

dependence or lock-in to any specific organization [Ref 5]. Postgres also exposes changes to downstream Change Data Capture (CDC) consumers making it suitable for decision urgency criteria.

Additionally, the Postgres ecosystem with its data store options, deployment options, and data analytics options is an ideal starting point for many AI use cases [Postgres for everything](#). Building on existing Postgres deployments is great investment protection and lowers incremental investment as well. As a side note, besides Postgres any other database with equivalent functionality may also be used.

There are 2 instances of Postgres:

- The first Postgres labeled “Source” proxies an operational database and is optimized for transactions with row-store. Built on Postgres15, this simulates an organization's live system of record data (example customer or inventory). Multiple instances of operational databases from diverse functions can be connected as future steps. Write ahead Log (WAL) is configured for change data capture.
- The second Postgres is labeled “Destination” is row-store tuned for bulk loads of data and dbt transformations, i.e., improving data quality. This implements the “Medallion” architecture - data processed from Bronze (raw data) to Silver (clean and deduplicated) and presentation. Dbt handles data quality progression and connects to MCP servers for natural language queries.

Both instances require their dedicated persistent volumes for storing application data but the volumes themselves are managed by Kubernetes. Two more Postgres exporter pods are created - one each for source and destination. These pods query statistics from Postgres instances, export them as Prometheus metrics and eventually to Grafana dashboards.

Standard Grafana based dashboards can be imported from [Postgresql Database - ID 9628](#). The metrics for source and destination databases can be filtered and viewed independently. The metrics show CPU, memory usage, file descriptors, database stats, lock tables, active sessions and transactions. In the next phase of testing, these metrics will be covered in more detail.

5.2 Apache Airflow

An open source platform designed to create, schedule and monitor batch workflows for many enterprise use cases - business ops, ETL/ELT, and MLOps, refer <https://airflow.apache.org/>. It allows development of complex end-to-end workflows with Directed Acyclic Graphs (DAG) using Python, built-in scheduling, retry policies, task level alerting, and idempotency. There is a Kubernetes friendly official Helm chart to define, install, and upgrade deployment [Ref 6].

In this POC, the following Airflow pods will be spun using [Airflow Architecture](#).

Airflow API server v3.2 architecture

- Serves as Airflow Web UI to visually track pipelines. The Airflow 3, which replaces legacy webserver, presents a user interface to visually inspect and track pipelines,

trigger DAGs, and debug task behavior. The UI is set up via Kubernetes NodePort that redirects to the browser IP port. It shows running, active, paused, successful, and failed DAGs, actions required, logs and history for a week.

Airflow DAG processor

- Directed Acyclic Graph processor. Constantly reads Python DAG files to update the Airflow UI with latest pipeline code.
- Parses DAG files and serializes them into the metadata database

Airflow PostgreSQL

- An internal metadata database that stores the state of tasks, DAGs, and records the history of jobs run and time.
- This requires a persistent volume managed by Kubernetes statefulsets. The official Helm chart requests ReadWriteMany (RWX) for logs volume so many pods can share it. However for this local deployment ReadWriteOnce (RWO) has been used which disables log persistence and logs live ephemerally on the pods.

Airflow Scheduler

- The brain of Airflow; it handles both triggering scheduled workflows and submitting tasks to the executor to run. The executor runs within the scheduler process.
- In this POC the executor checks with the scheduler and decides when to trigger Spark jobs and dbt transformations.

Airflow Statsd

- This acts as a metrics bridge. Airflow generates a ton of internal telemetry (e.g., msec for tasks, number of failed DAGs, utilization of scheduler). Airflow packages these metrics into statsd pod which translates them into a Prometheus friendly format. Grafana dashboards then display Airflow-specific data.

Airflow Triggerer

- Optional triggerer which executes deferred tasks in an asynchronous event loop. It consists of one main container and two sidecars. Here it runs background tasks like waiting for Spark jobs to finish without consuming CPU.
- It maintains logs that will be stored in persistent volume managed by Kubernetes statefulsets.

Airflow UI the main dashboard displays high level DAG status (Running, Active, Queued, and Failed) that can be filtered by multiple time periods. The specific DAGs as well as a list of DAGs in different states can be drilled-down.

5.3 dbt - Data Build Tool

Lightweight framework that helps write, test, document and run the data transformations inside Postgres analytics database to improve data quality for decisions [Ref 7].

dbt is ideal in this pipeline because the data is already resident in the analytics database, needs SQL based transformations, smaller (<10GB) data sets that fit in memory, and needs version control and testing for analytics.

Instead of running as a standalone pod, dbt is executed dynamically on-demand by Airflow (via the Cosmos library), spinning up sub-processes within the Airflow environment to run SQL models and shutting them down instantly upon completion. Hence there is no dedicated pod for dbt.

DBT handles progression through data quality layers. The naming convention evolves as the data moves through the layers:

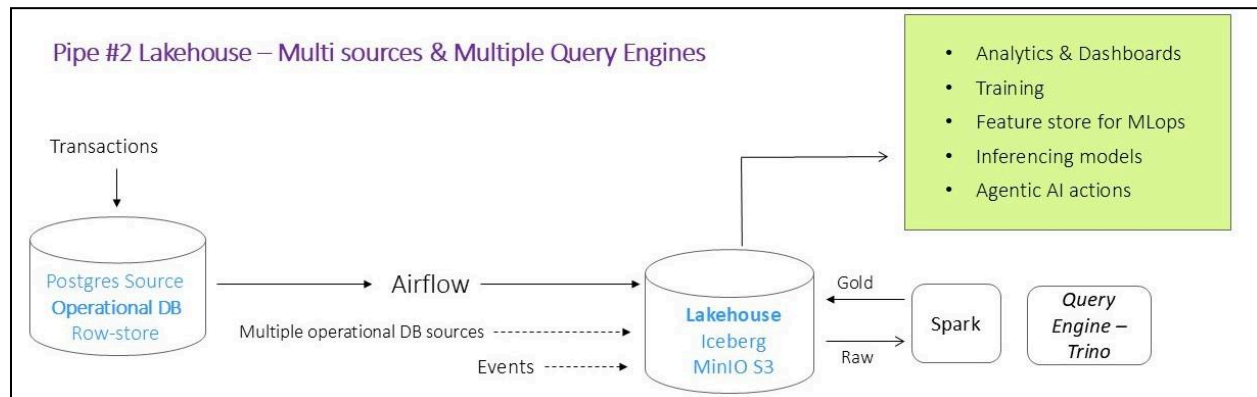
1. Bronze (Raw) & Silver (Cleaned): The tables and columns retain the exact same naming conventions as the operational Source database (e.g., orders, customer_id). This ensures absolute fidelity to the source system. Silver data may be used for training data via MLops or feature stores.
2. Gold quality data layer: Data used for analytics, inferencing, and agentic AI. The naming convention changes. The data will be modeled into a Star Schema of Dimensions and Facts, so tables take on standard prefixes like fact_ and dim_ (e.g., the raw orders table becomes fact_orders).

The entire translation map is strictly handled by dbt. Inside dbt's .sql and .yaml files, defines source column maps to Gold column. dbt automatically handles the lineage, so data engineers can trace any Gold column back to its original Source name.

5.4 Natural Language Queries

This provides the AI Assistant interface that allows business users to query data resident in Postgres analytics without depending on data analysts to run structured queries. Data query dashboard based on Custom Next.js application. The UI is set up via Kubernetes NodePort that redirects to the browser IP port. This will be covered in the next paper.

6. Infrastructure for Data Pipe #2 Lakehouse - Multiple Sources and multiple Query Engines



Airflow (described above) is used to transfer data from Postgres source operational database to Lakehouse - MinIO for lower cost object storage and Iceberg. Other data sources may also be fed to the lakehouse. “Events”, defined here as data that is not captured in any transactional database but affects decisions, may also be streamed to the lakehouse for analysis.

In this POC, Apache Spark is used for data analytics and data quality. However many different query engines from many sources may be used for high data accessibility and availability without compromising data quality.

6.1 Apache Iceberg and MinIO

A lakehouse is used as a centralized data lake with analytics capabilities, ACID, versioning, auditing, and more features needed for data quality and lineage. Apache Iceberg has built momentum for adoption and is supported by major vendors. It can ingest data from many sources via streaming or batch. Second Iceberg allows many query engines to read, write, and govern the ingested data without breaking consistency data

MinIO object storage includes Apache Iceberg tables. It is a massive data lake that is very economical for data capacity and cost but trades-off on decision urgency [Ref 8].

In this POC, Spark jobs write raw and processed data here as compressed Parquet files, organized using Apache Iceberg table format. This is an ideal staging ground for AI datasets for use by MLOps.

- Bronze layer: Raw data extracted from sources and is stored exactly as it arrived. It is stored as Parquet files in MinIO and staging tables. Pipe 2 consumes the bronze

parquet produced by Pipe 1's ingestion DAG. Data engineers access it to improve data quality and data scientists access for context for data lineage.

- Silver layer: Data cleaned and deduplicated by Apache Spark, stored as highly-performant Apache Iceberg tables in MinIO. This is ideal for MLOps training, feature stores, and large scale ad-hoc queries.
- Gold layer: Analytics-ready aggregations transformed by Spark and stored as high quality data layer for business intelligence and AI/agenting inferencing. Besides Spark (used in this POC) many query engines can access these layers.

Iceberg and MinIO require persistent volume that is managed by Kubernetes statefulsets.

Observability for object storage can be set up at 2 levels of visibility:

- The first view is a higher abstraction of data buckets. It is set up as Kubernetes NodePort that redirects to the browser IP port. Buckets are used to organize storage where each bucket holds an arbitrary number of objects. This allows data engineer to navigate the available datasets by data quality and availability. The deployment test script installs Bronze and Silver buckets.
- The second view is via Kubernetes observability of MinIO. This can be imported via Grafana UI [13502 MinIO Dashboard](#). It shows the infrastructure view - raw capacity, data usage growth, ingress/egress totals and rates, # of servers, CPU, memory, and IO usage. Some overlapping metrics of buckets and objects are also available.

6.2 Apache Spark 3.5

Apache Spark (<https://spark.apache.org/>) is used to build the Gold data layer directly inside Iceberg/MinIO.

Spark spins up one master pod and two of Spark worker pods. These pods provide the compute to write data back into MinIO as Iceberg tables. For massive datasets (100GB+) expected in Iceberg, it is easy to scale out by adding more Spark-worker pods to process data in parallel. These Kubernetes levers help where data for decision urgency is critical. Spark is ideal for complex AI/ML training workloads and transforming raw files at scale.

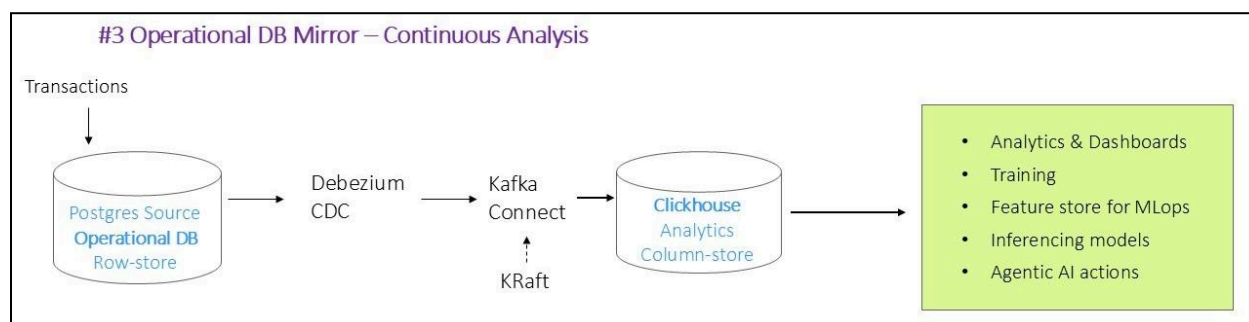
Spark master and workers are purely compute engines. They do their entire processing in RAM (memory) and write the final results directly to MinIO (S3). They are Kubernetes statefulsets but do not request persistent volume claims because they don't store data on disk via PVC.

The Spark UI is set up via Kubernetes NodePort that redirects to the browser IP port. The UI shows details of master and worker nodes, cores and memory used, and applications in execution and completed.

Distributed query engines may be used to query Iceberg data directly from respective BI tools. [Trino](#) query engine has been deployed and serves Iceberg Lakehouse over SQL. Trino is deployed on Kubernetes using the Helm chart. Once tested on a local machine or on-prem development cluster, Trino can be deployed to the production cloud for scale up.

Trino connects via its native Iceberg connector. Trino shares the JDBC catalog with Spark so both engines see identical tables; refer [Iceberg connector — Trino 482 Documentation](#).

7. Infrastructure for Data Pipe #3: Operational CDC Mirror - Continuous Analysis



7.1 Kafka Confluent

Kafka event streaming platform [Ref 9] is deployed to process data with higher urgency without slowing down caused by batching. It acts as a massive, high-speed message queue, transmitting data changes from source to destination databases. It decouples the source database from downstream consumers thereby maintaining its operational database performance.

Kafka also adds resiliency to the pipe - protecting against any downtime of destination Postgres or other downstream entities; Kafka securely stores the change data events until destination Postgres is available. Kafka requires persistent volume managed by Kubernetes statefulsets.

Kafka organizes the data change events in topics where each topic corresponds to the data category. Each topic can have one or many producers and consumers. For resiliency, events in Kafka are not deleted after consumption but remain until a pre-defined configurable maximum has been reached.

Kafka observability to view status and potential bottlenecks affecting data urgency is set up via Kubernetes NodePort that redirects to the browser IP port. AT high level displays topics, brokers, and consumers.

It shows a list of Debezium events under 4 Debezium topics which is Kafka's structure to organize events. Further topics can be distributed into partitions for higher scalability; topics can be replicated for fault tolerance. Lastly the events (or messages) can be read often by downstream Postgres consumers, they are not deleted immediately rather deleted after a configured retention period. The UI displays the number of partitions for each topic, the replication counts, along with the number of messages and size.

Brokers are agents assigned to servers/storage resources that transfer the change events from Postgres source to destination. UI lists the list of brokers, broker count, and their assignment to partitions. Lastly list Postgres destination that is a consumer is listed by number of members and topics.

7.2 Kafka Connect and Debezium

Debezium is a connector to source databases and resides inside the Kafka Connect pod. Debezium reads the write-ahead log (WAL) of source Postgres for any inserts/updates/deletes. When it detects changes, it instantly grabs the changes and converts them into events and streams it to Kafka.

Kafka Connect is completely stateless on persistent storage. It saves all of its configurations, states, and offsets directly into Kafka topics over the network. It does not need a local storage to save anything permanently. This is unlike many other Kubernetes statefulsets that are bound to persistent volumes via claims (PVC).

7.3 Strimzi

Strimzi, an open-source CNCF project, provides a way to run an Apache Kafka cluster on Kubernetes in various deployment configurations. It is based on the Kubernetes operator pattern to deploy, run, and manage collections of Kafka brokers on Kubernetes. It provides a declarative way to manage Kafka clusters. Developers can describe what they want in a configuration file, and Strimzi handles deployment, upgrades, and maintenance. This promises fewer manual interventions and more predictable operations.

7.4 KRaft

KRaft, is Kafka's built-in metadata management system, replacing Zookeeper for scalability and reliability. KRaft provides more flexibility to configure Kafka server roles. In this POC, Kafka server is configured as a combined broker and controller. Second, the Raft based protocol manages the metadata in a more reliable and efficient way.

For background, Zookeeper was the high-availability traffic controller to manage Kafka metadata and broker coordination and to keep the distributed Kafka system stable and consistent. Zookeeper has been deprecated [Ref 10 and 11].

7.5 ClickHouse

[ClickHouse](#) is an open-source, column-oriented database built for real-time analytics. Kafka connects to ClickHouse's built-in Kafka engine. ClickHouse works the data transfers and analytics independent of the operations database thereby not compromising the transactional database performance. Clickhouse reflects the latest state of operational data using last-write wins, dedupe, and delete filtering via RelaceMergeTree. This pipe delivers a hybrid transaction and analytics processing (HTAP) system. For now data quality layers are in pipes #1 and #2

8. Observability for pipelines

Observability is the first pillar of data pipeline architectures that is available from many different levels. In cloud native eco-system of distributed infrastructures described above, observability is critical for setup and debug of the data **pipelines**. Second, the observability and sanity checks of the data flowing through the pipelines - from data timelines (aka Freshness) and anomaly alerting. Implemented are a custom provisioned Grafana "Data Platform Health" dashboard, CDC consumer-lag metrics, source-freshness tracking, a metrics anomaly z-score, and five alert rules routed through Alertmanager.

Third observability monitors resources used and bottlenecks allowing course correction to meet data urgency SLAs. This set of dashboards offer one more layer of visibility at the hardware layer of processors, memory, storage, network, and IO as consumed by Kubernetes pods. Observability will be covered in more detail in the future as more test cases are run.

8.1 Node Exporter

These run in the background, scraping memory, CPU, storage, network and other health metrics from all the other pods and relays to Prometheus that can be charted by Grafana. Hardware resources that may be causing bottlenecks thereby compromising decision urgency can be identified for corrective action.

Auto-provisioning is enabled in Grafana YAML with one standard dashboard. When Grafana starts, it automatically scans `/var/lib/grafana/dashboards/` and loads any JSON files found there. A placeholder dashboard is pre-loaded there that tells you exactly how to import the full dashboard. Many standard dashboards are available for import from grafana.com/grafana/dashboards. Specific to Node exporter check [Node Exporter Full - ID 1860](#).

8.2 Cilium and Hubble

CNCF's [Cilium](#) has the advantage of monitoring the infrastructure from Linux kernel's eBPF. It displays CPU, virtual and resident memory usage per server node, # of API calls, API call latencies, and many more.

[Hubble](#) built on top of Cilium enables visibility into communications between services. It displays flow types, flows processed per node, L7 flow distribution, network operations success/drops with reasons for drops, protocol usage, and port distribution.

8.3 Prometheus

Time series database that collects, stores, and relays metrics from above to Grafana. Prometheus UI can be accessed after mapping its ClusterIP as a NodePort. From this UI, the raw stats gathered can be probed using PromQL code. Typically used to trace data transfer

issues from statistics gathering engines to Grafana dashboards. The operational status of the database itself can also be checked via its UI.

8.4 Grafana

Grafana reads metrics data from Prometheus and displays it as real-time panels and charts that can be visualized in many different ways, over many time windows, and drill downs. The UI is set up via Kubernetes NodePort that redirects to the browser IP port.

Many standard dashboards at the cloud infrastructure and application level can be imported from <https://grafana.com/grafana/dashboards/>; the numbers can be used in Grafana panels UI to import respective dashboards.

- [Kubernetes Cluster Monitoring via Prometheus - ID 315](#) (refer Cloud Native section)
- [Postgresql Database - ID 9628](#) (refer Postgres section)
- [Minio Dashboard - ID 13502](#) (refer MinIO section)

9. Security

For security In this first phase of POC, the AI assistant uses a read-only database role and accepts only single SELECT statements. The observability UIs (including Kafka, Grafana, dashboards) require login authorization. Kubernetes deployments generate random credentials instead of defaults. Up to 16 blocking Continuous Integration (CI) checks gate every change. They include secret scanning, dependency audits, and static analysis. These protect from tampering, vulnerabilities, and malicious code. More will be added in future phases.

10. Test case and Use case

One detailed test case comes as part of the deployment script. This tests the basic functionality of the deployed infrastructures. The test triggers all three data pipeline flows simultaneously. It injects simulated e-commerce records, verifies that Debezium/Kafka instantly captures the CDC events (Pipe #3), and triggers Airflow to micro-batch the data to Bronze, compact it into Silver data layer on Iceberg via Spark, and model it into Gold data layer via dbt (Pipes #1 & #2).

The use case source data tables are defined in `airflow/dags/config/pipelines.yml` manifest file — the single source of truth. The ingestion DAG reads it directly; the ClickHouse mirror schema and Debezium connector are generated from it. Continuous Integration (CI) fails if the generated files drift from the pipeline manifest thereby preventing silent drifts.

11. Summary and Next Steps

This Proof of Concept (POC) sets the stage for three data pipelines mapped for organization's specific use cases and needs. The first two criteria are data quality and decision urgency to make decisions. The next criteria is the extent of access to data with the organization - ranging from PostgreSQL to Apache Iceberg. This is set up using open-source, cloud native infrastructures that simplify deployment, scaling, and monitoring of resources. The many infrastructure elements required for the three pipelines are described.

We're investigating Postgres destination in pipeline #1 changed to a column store for data analytics reporting, business intelligence tools, training AI models, and inference AI models. This will be an option for organizations who continue to build on Postgres skill sets.

The next pillar is **performance**. More comprehensive data transfer tests over longer periods of time will be run - individually testing each of the 3 pipelines. This will be followed by benchmarks for transaction processing and analytics processing. For example pg_bench and TPC-C for OLTP (source database) and TPC-H for OLAP (destination databases or lakehouses). Our goal is to publish the performance data as reference for data pipeline architects. The performance tests will also populate the observability panels in a more meaningful way. Second, the natural language interfaces to DBT via MCP can be validated more robustly.

MCP servers for PostgreSQL and Apache Iceberg allow AI tools and agents to connect to the respective data stores to perform queries using natural language and also service/initiate agentic AI actions.

Machine learning models need high quality data, in a timely fashion, versioned, and reproducible for feature stores, experiment tracking, model registries, and automated retraining pipelines [Ref 12]. A harness from Postgres destination or Apache Iceberg to MLops cycle will be provided. Models created through the MLops cycle or from other sources can be made available for inferencing on real time data that leads to up-to-date recommendations for urgent decisions and actions.

Importantly sample test cases from use cases in banking, finance, health, life sciences, manufacturing, insurance [Ref 13] and logistics will be tested. Focus will be on the cycle: Data (quality) - Analytics - Insights - Recommendations - Decisions - Actions (speed up urgency with AI only where it makes sense).

In the long term continue to address the key pillars of data governance, security, reliability, resiliency, and scaling multiple data sources and destinations, all built around cloud native, data engineering, and open-source infrastructures [Ref 14 - 20].

Authors

- **Pradeep Kalluri** is a Data Engineer with experience building production data platforms at NatWest, Accenture, and Dpoint. Specialized in cloud-native architectures, real-time processing with Kafka and Spark, and data quality frameworks. Published technical writer on Medium, sharing practical lessons from production systems. Passionate about making data platforms reliable and trustworthy. <https://www.linkedin.com/in/pradeepkalluri/>
- **Raghu Shankar** is a product management and engineering professional. Entrepreneur incubating cloud native and semiconductor architectures, <https://www.linkedin.com/in/raghu-shankar/>

References & Related Readings

1. "From Raw to Refined: Data Pipeline Architecture at Scale", Pradeep Kalluri via Medium, Nov 2025
2. "Batch vs Streaming – Choosing the right architecture", Pradeep Kalluri, LinkedIn, Feb 2026
3. "About dbt Model Context Protocol (MCP) server",
<https://docs.getdbt.com/docs/dbt-ai/about-mcp?version=2.0&name=Fusion>
4. "Introducing the dbt MCP Server – Bringing Structured Data to AI Workflows and Agents", Blog by Jason Ganz, DBT Labs, April 2025
<https://docs.getdbt.com/blog/introducing-dbt-mcp-server?version=1.12>
5. "[Just Use Postgres for Everything](#)", Stephan Schmidt, updated, December 13, 2025
6. [Beginner's Guide to contributing to Airflow - Kalluri](#)
7. "Query Tools Criteria", LinkedIn, Pradeep Kalluri, Jan 2026
8. Minio AIStor Documentation, <https://docs.min.io/aistor/>
9. Kafka documentation: <https://kafka.apache.org/43/getting-started/introduction/>
10. Why is Kafka replacing Zookeeper with KRaft:
<https://www.confluent.io/learn/zookeeper-kafka/#apache-kafka-without-zookeeper-introduction-to-kraft>
11. KRaft vs Zookeeper: <https://kafka.apache.org/42/getting-started/zk2kraft/>
12. "Modern Data Engineering Meets MLOps: Building the Backbone of AI-Driven Products", Anchit Gupta, Learn Data Now, via Medium, April 2026
13. "The evolution of cloud-native data platforms for insurance analytics: A Technical Review", A.R. Malipeddi, WJAETS, May 2025
14. "Dashboards Are Dead. The Future Is Decision Execution", Cloud with Azeem, July 2026
15. "Data Engineering System Design: Building a CDC-Based Pipeline from OLTP to a Lakehouse", Sarath Sagi, Medium, Dec 2025
16. "The next evolution of Data Engineering: From Data Pipelines to Context Pipelines", Think Data, Medium, June 2026.
17. "9 Reasons Why PostgreSQL is All You Need", Oz, Medium, Dec 2025
18. "True enterprise sovereignty is more approachable than ever, thanks to K8s-powered cloud-neutral PostgreSQL", EnterpriseDB, TheNewStack, April 2026
19. "The Data Engineering: How Modern Data Infrastructure is Powering the AI Revolution", Nithin Narla, Towards AI, Medium, April 2026
20. "The Real Reason Data Teams Are Quietly Replacing Their Data Warehouses", Saurav Singh, Medium, May 2026